

Program 6 Report

Title:
Reading Data from an SPI Flash Memory Device

Author:
Magnus Arnfinn Aubell

Date: October 18, 2025

Subject: EECE375

1 Purpose

Serial Peripheral Interface (SPI) is the de facto standard for communication between digital chips. [1] It bases itself on a slave-master architecture with information sent between the master and slave done serially. The master and slave have usually have common inputs/outputs like SPI clock *SCLK*, reset *RESET_N*, slave/chip select *SS_N*, master-out-slave-in *MOSI* and master-in-slave-out *MISO*. Note that for this program some of these are active-low indicated by "*_N*".

The purpose of this program is to make a master SPI. The SPI will use the normal read command for a 4 byte address for the S25FL128S chip (found in [2]), 0x03. Afterwards it will send the address of the data. For this program, the address 0x857140 will be used.

To test the SPI master module, the testbench will include a dummy flash-memory slave. This will simply return the data 0x5714 if the read and address is sent out correctly from the SPI master module.

Further specifications for the program can be found in [3], which the purpose section was largely based.

2 Verilog code

The Verilog code for the SPI master module and testbench are each shown in subsections 2.1 and 2.2, respectively.

2.1 SPI Master Module

```
1 // Serial Peripheral Interface Module
2 // Made by Magnus Arnfinn Aubell
3 // Last modified 17-10-2025
4 // SPI master structure based on description from following source:
5 // [1] Wikipedia contributors. Serial peripheral interface. https://en.wikipedia.org/wiki/Serial\_Peripheral\_Interface, 2025. Accessed: 2025-10-17.
6 // Note: CPOL = CPHA = 0 when compared to standard SPI. Option to change not
   added, but possible by modifying pos-/negedge
7 `timescale 1ns / 1ps
8
9 module SPI(
10     input CLK_IN,
11     input RESET_N,
12     input MISO,
13     output reg SCLK,
14     output SS_N,
15     output reg MOSI,
16     output [15:0] DATA
17 );
18
19     reg [32:0] Output;
20     wire sent; //Bool used to stop sending when done
21     reg [17:0] data_temp; //Temp data used to store data, bit to skip first
   negedge sample and end of shifting bit 1
22
23     assign SS_N = ~RESET_N;
24     assign sent = (Output == 33'h100000000);
25     assign DATA = data_temp[15:0];
26
27     initial begin
28         data_temp = 0;
29         SCLK = 0;
30         Output = {8'h03, 24'h857140, 1'b1}; //read command + address + bit for
   skipping first sample negedge + bit indicating end
31         data_temp = 18'b1; // bit indicating end
32         MOSI = 0;
33     end
34
35     always @(posedge CLK_IN) begin //CLK halfer
36         if(RESET_N) begin
37             SCLK <= ~SCLK;
38         end else begin //Idle state
39             SCLK <= 0;
40         end
41     end
42
43     always @(negedge SCLK) begin // Sends on negedge
44         if(RESET_N & !sent) begin
45             MOSI <= Output[32];
46             Output <= Output << 1;
47         end
48     end
49
```

```
50     always @(posedge SCLK) begin //Sample on posedge
51         if(RESET_N & data_temp[17] != 1 & sent) begin
52             data_temp <= {data_temp[16:0], MIS0};
53         end
54     end
55 endmodule
```

2.2 SPI Master Testbench

```
1 // Serial Peripheral Interface Module
2 // Made by Magnus Arnfinn Aubell
3 // Last modified 17-10-2025
4 // SPI slave structure based on description from following source:
5 // [1] Wikipedia contributors. Serial peripheral interface. https://en.wikipedia.org/wiki/Serial\_Peripheral\_Interface, 2025. Accessed: 2025-10-17.
6 // Note: CPOL = CPHA = 0 when compared to standard SPI designs. Option to
   // change not added, but possible by modifying pos-/negedge
7
8 `timescale 1ms / 1ps
9
10 module SPI_tb(
11 );
12     reg clk;
13     reg reset_n, miso;
14     wire sclk, ss_n, mosi;
15     wire [15:0] data;
16     reg [31:0] dummy_memory_in;
17     reg [15:0] dummy_memory_data;
18
19     initial begin
20         miso = 0;
21         dummy_memory_in = 0;
22         dummy_memory_data = 16'h5714;
23         clk = 0;
24         forever #10 clk = ~clk; //50Hz
25     end
26
27     SPI spi( // UUT
28         .CLK_IN(clk),
29         .RESET_N(reset_n),
30         .MISO(miso),
31         .SCLK(sclk),
32         .SS_N(ss_n),
33         .MOSI(mosi),
34         .DATA(data)
35     );
36
37
38     // Dummy memory begin
39     always @(posedge sclk) begin // Sample on posedge
40         if (~ss_n & dummy_memory_in[31:24] != 8'h03) begin
41             dummy_memory_in <= {dummy_memory_in[30:0], mosi};
42         end
43     end
44
45     always @(negedge sclk) begin // Send on negedge
46         if (dummy_memory_in == {8'h03, 24'h857140} & ~ss_n) begin
47             miso <= dummy_memory_data[15];
48             dummy_memory_data <= dummy_memory_data << 1;
49         end
50     end
51     //Dummy flash memory end
52
53     initial begin
54         //checks normal function
55         reset_n = 0;
56         #40
57         reset_n = 1;
58     end
59
```

```
60 | endmodule
```

3 Waveforms

The waveforms are shown in figures 1-???. The figures divide the waveforms into different part of the SPI-Flash memory communication. The initial communication sent by the SPI master is shown in figure 1.

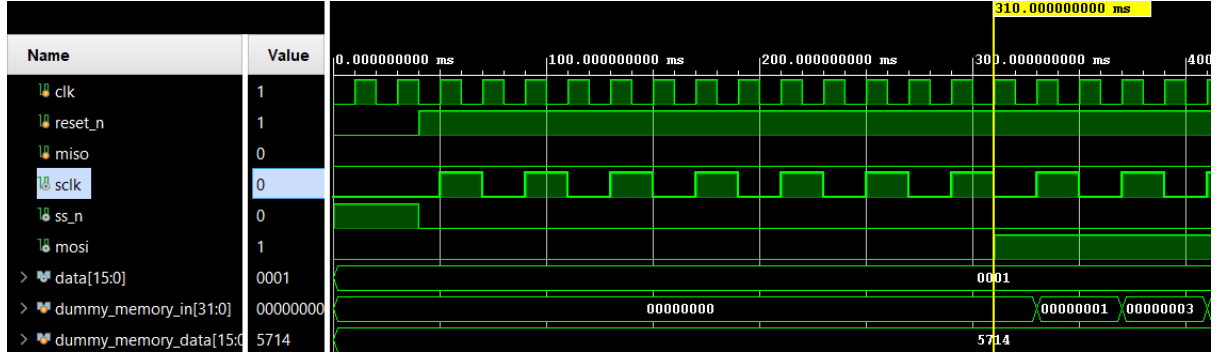


Figure 1: Waveforms from beginning of transaction

The yellow line in figure 1 show the first change in *mosi*. Note that this is 6 low values after the change in *reset_n*, which matches the first 6 low values in the read command, 0x03. After the yellow line, this value (0x03) can be seen as being sampled on the *dummy_memory_in* row. Note that this sampling happens on the negative edge of *sclk*.

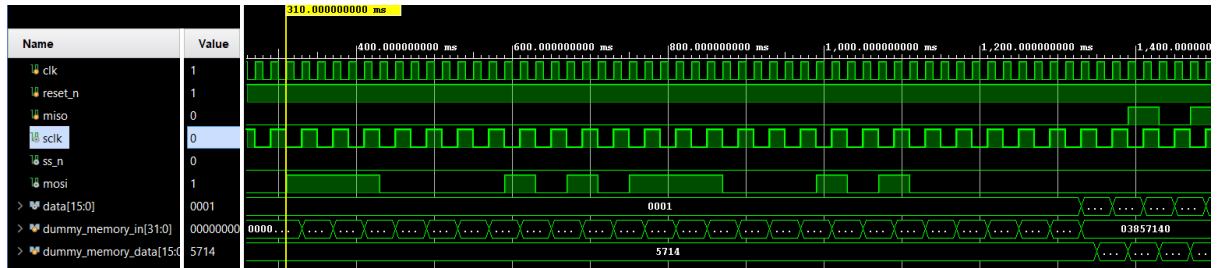


Figure 2: Waveforms showing SPI master transferring instruction and address to the dummy memory.

In figure 2 the rest of the read instruction and the address 0x857140 can be seen transferred through *mosi*. At the end of the transaction, towards the right of the figure, this value can be seen in the *dummy_memory_in* row. Afterwards, a change in both SPI master's local *data* and the *dummy_memory_data* can be seen, where the latter is transferring it's most significant bit to the former. Note that *data* changes once before *dummy_memory_data* has started to be transferred. This is taken into account in the design, and this bit is later discarded.

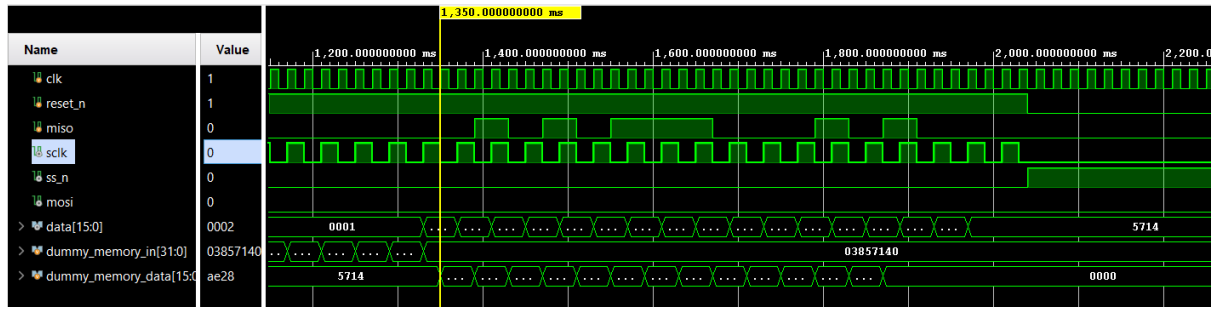


Figure 3: Waveforms showing the data from address 0x857140 being correctly transferred to the SPI master's internal *data*

Figure 3 shows *dummy_memory_data* shifting and SPI master's internal *data* getting these values. At the end all values have been shifted and the SPI master has received the value 0x5714 in *data*.

4 Synthesis

The full synthesis is shown in figure 4. The synthesis largely utilizes flip-flops, reflecting the verilog code's heavy use of left shift to transfer data.

On the left side near the *CLK_in* input, the register "SCLK_reg" and it's function as a clock divider for the *SCLK* signal can be observed. Note how it goes to an active low when *RESET_N* is low as LUT2 is an and gate.

All the registers for the variable *Output_reg* can be observed as being set up in a left-shift manner, as the output of the previous value is sent into the register of the next value. Towards the middle of the design the same can be observed for *data* in the *data_temp* registers.

Lastly, note that some of the *data_temp* and *Output* registers are placed toward the right end which might seem weird. This is due to them being used in combinatory logic toward the end of the circuit which stop the shifting of *data* and *Output* when they are empty.

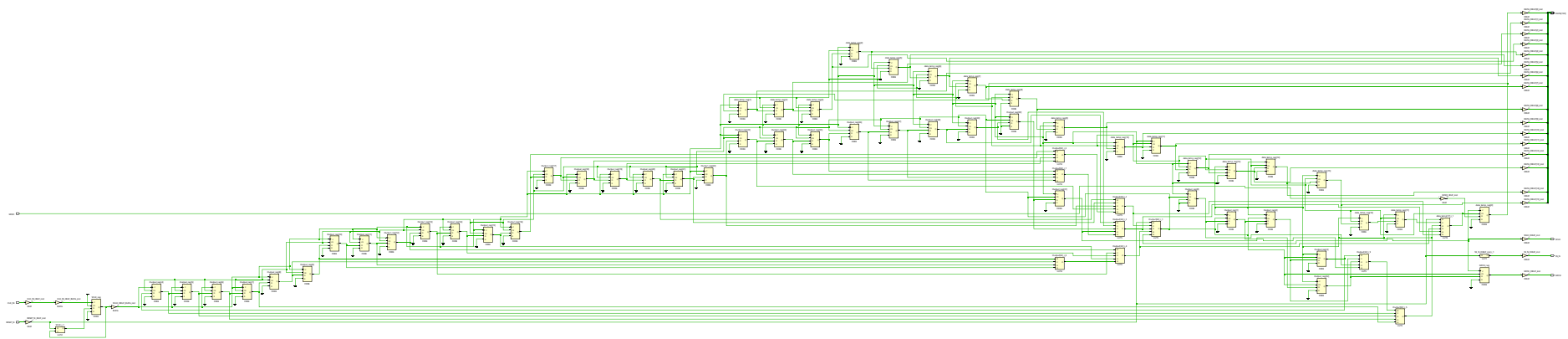


Figure 4: Full synthesis of master SPI.

5 Discussion

The results were as expected. The SPI that was designed is only available in the $CPOL = CPHA = 0$ as described by [1]. This means that $SCLK$ rests at 0 when the SPI master is idle. It also means that on the negative edge of $SCLK$, SPI master send data to MOSI, and samples on $SCLK$'s positive edge.

Lastly, note that capital letters were used for input/output letters, and smaller case letters used for local variables. Lower letter variable names are therefore used in the waveforms discussion as these are the local variables of the testbench.

References

- [1] Wikipedia contributors. Serial peripheral interface. https://en.wikipedia.org/wiki/Serial_Peripheral_Interface, 2025. Accessed: 2025-10-17.
- [2] Infineon Technologies. S25fl128s 128 mb (16 mb)/256 mb (32 mb) fl-s flash spi multi-i/o, 3.0 v datasheet, 2025. Accessed: 2025-10-17.
- [3] Sung Gu Lee. Reading data from an spi flash memory device. EECE375, 2025. Accessed: 2025-10-17.